OPERATING SYSTEM (CT 656)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 9: System Administration

Syllabus Hours: 4 hrs | Marks Weight: Part of 20 marks (Chapters 6, 8, 9)

Complete Exam-Ready Answers | All Past Questions Covered (2073–2082)

CHAPTER 9 SYLLABUS TOPICS

- 9.1 Administration Tasks (What a sysadmin does daily)
- 9.2 User Account Management (adduser, userdel, passwd, groups, /etc/passwd, /etc/shadow)
- 9.3 Start And Shutdown Procedures (init levels, systemd, shutdown, reboot)
- 9.4 Setting up Operational Environment for a New User
- 9.5 AWK tool — Pattern scanning and processing language
- 9.6 Search tools — grep, find, locate
- 9.7 Sort tools — sort, uniq, wc
- 9.8 Shell scripts — sh/bash scripting, variables, control flow
- 9.9 Make tool — Automated build system

Prepared by: Santosh Kushwaha (ACE079BCT)

SECTION 1: System Administration — Definition, Tasks & Roles

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2080 Chaitra, 2079 Ashwin, 2079 Jestha, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Magh

Q1. Define a System Administrator. What are the key technical and soft skills required? List and explain the roles and responsibilities with examples. [1+5]

Exam Ref: 2082 Kartik [1+5], 2081 Ashwin, 2080 Chaitra, 2079 Ashwin

Answer:

Definition of System Administrator

A System Administrator (SA or SysAdmin) is an IT professional responsible for managing, configuring, maintaining, and ensuring the reliable operation of computer systems, servers, networks, and IT infrastructure of an organization. They are the guardians of the IT environment — keeping systems secure, available, and performing optimally.

Roles and Responsibilities of a System Administrator

Responsibility	Description	Example
User Account Management	Create, modify, and delete user accounts; assign permissions and group memberships	adduser alice; usermod -aG sudo alice
System Installation & Configuration	Install and configure OS, software packages, services	Installing Apache, configuring nginx.conf
Security Management	Set up firewalls, manage ACLs, monitor security logs, apply patches	ufw enable; fail2ban; sudo apt update
Backup & Recovery	Implement backup policies; test and perform disaster recovery	rsync backups; restore from /backup
Performance Monitoring	Monitor CPU, memory, disk, network; identify bottlenecks	top, htop, iostat, netstat, sar
Network Administration	Configure IP addressing, DNS, DHCP, routing, VPN	Setting up /etc/network/interfaces
Storage Management	Manage disk partitions, RAID, file systems, quotas	fdisk, lvm, df -h, quota
Process & Service Management	Start/stop/restart services; manage daemons and scheduled jobs	systemctl restart nginx; crontab -e
Documentation	Maintain records of configurations, changes, incidents	IT runbook, change log, SOP documents
Troubleshooting	Diagnose and resolve hardware and software failures	Analyzing /var/log/syslog, dmesg

Technical vs Soft Skills

Technical Skills	Soft Skills
Linux/Unix/Windows Server administration	Problem-solving and analytical thinking
Networking (TCP/IP, DNS, DHCP, VPN, firewall)	Communication — explaining technical issues to non-technical staff
Scripting (Bash, Python, PowerShell)	Time management — handling multiple incidents simultaneously
Security (cryptography, authentication, ACL)	Documentation — keeping accurate, up-to-date records
Virtualization (VMware, Docker, KVM)	Patience — dealing with user requests and repeated problems
Cloud platforms (AWS, Azure, GCP)	Attention to detail — one wrong config = system outage

---

Q2. What is the significance of system administration? Describe the roles and responsibilities of a system administrator to keep the system updated and efficient. Explain with an example. [3+5]

Exam Ref: 2075 Bhadra, 2074 Bhadra [3+5]

Answer:

Significance of System Administration

System administration is critical for any organization relying on IT infrastructure because:

- Availability: SA ensures systems are up 24/7 — downtime costs money and reputation
- Security: SA protects systems from unauthorized access, data breaches, and cyberattacks
- Performance: SA optimizes system resources so users get fast, reliable service
- Continuity: SA maintains backups and disaster recovery plans
- Scalability: SA plans and implements infrastructure growth as the organization expands

Example — Nepal Telecom SA Scenario (from 2079 Ashwin exam):

A SA at Nepal Telecom is responsible for:

- Managing 10,000+ user accounts for employees and customers
- Ensuring 99.9% uptime of core network services (DNS, billing, customer portal)
- Configuring firewalls to block unauthorized external access to internal servers
- Automating nightly backups of customer databases to offsite storage
- Responding to security incidents within 1 hour as per SLA

SECTION 2: User Account Management & Start/Shutdown Procedures

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2076 Baisakh

Q3. Explain User Account Management in Linux/Unix OS. What files are involved? How does an administrator set up an environment for a new user? [5]

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin

Answer:

Key Files for User Management in Linux

File	Content	Example Entry
/etc/passwd	User account info: username, UID, GID, home, shell	alice:x:1001:1001:Alice:/home/alice:/bin/bash
/etc/shadow	Encrypted passwords + password policy (expiry, min/max age)	alice:6salt$hashedpwd:19000:0:99999:7:::
/etc/group	Group definitions: group name, GID, members	developers:x:1002:alice,bob,charlie
/etc/sudoers	Users/groups allowed to run commands as root	alice ALL=(ALL:ALL) ALL
/home/username/	User's home directory — personal files, shell config	~/.bashrc, ~/.profile, ~/Desktop

Common User Management Commands

Command	Purpose	Example
useradd / adduser	Create new user account	sudo adduser alice
userdel	Delete a user account	sudo userdel -r alice (-r removes home)
usermod	Modify existing user (groups, shell, etc.)	sudo usermod -aG sudo alice
passwd	Set/change user password	sudo passwd alice
groupadd / groupdel	Create/delete group	sudo groupadd developers
id	Show user's UID, GID, groups	id alice
who / w	Show logged-in users	who; w
su / sudo	Switch user / run as root	su - alice; sudo apt install nginx
chown / chmod	Change file owner / permissions	chown alice:alice file.txt; chmod 755 script.sh

Setting Up Environment for a New User — Step by Step

```
# Step 1: Create account
sudo adduser alice
```

```
# Step 2: Set password policy
sudo chage -m 7 -M 90 -W 14 alice

# Step 3: Add to appropriate groups
sudo usermod -aG developers,sudo alice

# Step 4: Configure home directory
cp /etc/skel/.bashrc /home/alice/
cp /etc/skel/.profile /home/alice/

# Step 5: Set permissions
chown -R alice:alice /home/alice
chmod 700 /home/alice

# Step 6: Verify
id alice && ls -la /home/alice
```

Q4. Explain the Start and Shutdown Procedures of a Linux/Unix system. What are runlevels (init levels) and systemd targets? [4]

Exam Ref: 2081 Ashwin, 2080 Ashwin, 2078 Chaitra

Answer:

Linux System Boot Process

- 1. BIOS/UEFI: Power-on self-test (POST); finds bootable device
- 2. Bootloader (GRUB2): Loads kernel into memory; shows boot menu
- 3. Kernel Initialization: Detects hardware; mounts root filesystem; starts init/systemd
- 4. init/systemd (PID 1): First process; initializes system services based on target/runlevel
- 5. Login: Display manager or text login prompt presented to user

Runlevels (SysV init) vs systemd Targets

Runlevel	systemd Target	Description	Usage
0	poweroff.target	Halt / Power Off system	sudo init 0 / shutdown -h now
1	rescue.target	Single-user / Rescue mode — root only, no network	Emergency maintenance
2	(Ubuntu default)	Multi-user without NFS (Debian-based default)	Normal mode in Debian/Ubuntu
3	multi-user.target	Multi-user, network enabled, no GUI	Servers without desktop
4	(undefined)	User-defined	Custom use
5	graphical.target	Multi-user + network + GUI (X11/Wayland)	Desktop workstations
6	reboot.target	Reboot system	sudo init 6 / reboot

Shutdown Commands

```
shutdown -h now          # Halt immediately
shutdown -h +10 'System going down' # Halt in 10 minutes with warning
shutdown -r now          # Reboot immediately
poweroff                 # Power off
reboot                   # Reboot
systemctl poweroff       # systemd power off
systemctl reboot         # systemd reboot
```

SECTION 3: Shell Scripts, AWK Tool, Search & Sort Commands

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2080 Chaitra, 2079 Chaitra, 2079 Ashwin, 2075 Bhadra, 2074 Bhadra

Q5. What is Shell Programming? Write a shell script to find factorial of a number. Explain variables and control structures in shell scripting. [2+5]

Exam Ref: 2082 Kartik, 2080 Chaitra, 2079 Chaitra

Answer:

What is Shell Programming?

Shell programming (shell scripting) is the process of writing a series of commands for a shell to execute as a program. Shell scripts automate repetitive administrative tasks, batch processing, and system management operations.

Shell Script Structure:

```
#!/bin/bash          # Shebang — specifies interpreter
# Comment line
variable=value       # Variable assignment (no spaces around =)
echo $variable       # Print variable
read var             # Read user input
```

Shell Variables

Variable Type	Description	Example
User-defined	Created by programmer	name="Alice"; echo $name
Environment	System-wide variables	$PATH, $HOME, $USER, $SHELL
Positional Params	Arguments passed to script	$0=scriptname, $1=first arg, $#=count
Special	Built-in special meanings	$? = exit status, $$ = PID, $! = last bg PID

Shell Script: Factorial of a Number

```
#!/bin/bash
# Program to calculate factorial

echo -n "Enter a number: "
read n
```

```
factorial=1
i=1

while [ $i -le $n ]
do
    factorial=$((factorial * i))
    i=$((i + 1))
done

echo "Factorial of $n = $factorial"
```

Output:

```
Enter a number: 5
Factorial of 5 = 120
```

Control Structures in Shell

Structure	Syntax / Example
if-elif-else	if [condition]; then ... elif [condition]; then ... else ... fi
for loop	for i in 1 2 3 4 5; do echo $i; done OR for ((i=0;i<10;i++)); do ... done
while loop	while [condition]; do ... done (used in factorial above)
until loop	until [condition]; do ... done (runs until condition is TRUE)
case	case $var in pattern1) cmds;; pattern2) cmds;; esac
Functions	myFunc() { echo "Hello $1"; } myFunc Alice

Q6. What is the AWK tool? Explain its use with examples. How does it differ from sed and grep? [5]

Exam Ref: 2082 Kartik, 2080 Chaitra, 2079 Ashwin

Answer:

AWK Tool — Introduction

AWK is a powerful text processing language/tool used for pattern scanning and data extraction. Named after its creators: Aho, Weinberger, and Kernighan (1977). AWK processes text files line by line, splitting each line into fields.

AWK Basic Syntax:

```
awk 'pattern { action }' filename
awk -F':' '{ print $1, $3 }' /etc/passwd    # -F sets field separator
```

AWK Concepts

Concept	Explanation
Records	Each line is a record (NR = record number)
Fields	Each word/column in a line is a field ($1, $2, ..., $NF = last field)
Field Separator	FS variable (default: whitespace); set with -F option

Built-in Variables	NR (line number), NF (num fields), FS (field sep), RS (record sep), OFS, ORS
BEGIN block	Executed before processing any input: awk 'BEGIN { print "Header" }' file
END block	Executed after all input: awk 'END { print NR " total lines" }' file
Pattern	Regular expression or condition to match lines: /ERROR/ { print }

Practical AWK Examples

```
# Print all usernames from /etc/passwd (field 1, separator ':')
awk -F':' '{ print $1 }' /etc/passwd

# Print lines where salary (field 3) > 50000
awk -F',' '$3 > 50000 { print $1, $3 }' employees.csv

# Count number of lines in a file
awk 'END { print NR }' data.txt

# Sum of values in column 2
awk '{ sum += $2 } END { print "Total:", sum }' data.txt

# Print lines matching pattern
awk '/ERROR/ { print NR": "$0 }' /var/log/syslog

# BEGIN and END blocks
awk 'BEGIN { print "Name  Score" }
     { print $1, $2 }
     END { print "Done" }' scores.txt
```

AWK vs grep vs sed Comparison

Feature	grep	sed	awk
Primary Use	Search/filter lines matching a pattern	Stream editor: find and replace text	Data extraction and processing
Field Awareness	No — treats line as one string	No — line-oriented	Yes — splits into fields ($1, $2...)
Computation	No	No	Yes — arithmetic, counting, sums
Typical Task	Find ERROR lines in log	Replace 'old' with 'new' in file	Compute salary total from CSV
Example	grep 'ERROR' syslog	sed 's/old/new/g' file.txt	awk '{ sum+=$2 } END { print sum }'

Q7. What is group policy? What can we do with AWK? Explain. Explain the search and sort tools available in Unix/Linux. [2+3+5]

Exam Ref: 2080 Ashwin, 2079 Chaitra

Answer:

Group Policy

Group Policy is an administrative feature that defines rules and settings applied to users and computers within a group or organizational unit (OU). In Linux, group policies are managed through:

- /etc/group — defines groups and their members
- PAM (Pluggable Authentication Modules) — enforces password policies, login restrictions
- sudo rules in /etc/sudoers — controls who can run which commands as root
- LDAP/Active Directory — enterprise group policy management for large organizations

Search Tools in Unix/Linux

Tool	Description	Common Usage
grep	Search for a pattern (regex) within files; print matching lines	grep 'error' syslog; grep -r 'TODO' ./src
grep -i	Case-insensitive search	grep -i 'error' syslog
grep -v	Invert match — print non-matching lines	grep -v '^#' /etc/hosts
grep -n	Show line numbers with matches	grep -n 'main' program.c
find	Search for files/directories by name, type, size, time	find /home -name '*.txt' -mtime -7
find -exec	Execute command on each found file	find . -name '*.log' -exec rm {} \;
locate	Fast database-based file search (uses updatedb)	locate passwd; sudo updatedb
which	Find executable location in PATH	which python3 → /usr/bin/python3
whereis	Find binary, source, and man pages	whereis nginx

Sort Tools in Unix/Linux

Tool	Description	Example
sort	Sort lines of text alphabetically or numerically	sort names.txt; sort -n numbers.txt
sort -r	Sort in reverse order	sort -r -n scores.txt
sort -k	Sort by specific column/key	sort -k2 -n employees.txt (sort by column 2)
sort -u	Sort and remove duplicates	sort -u fruits.txt
uniq	Report or remove duplicate adjacent lines (usually after sort)	sort names.txt | uniq
uniq -c	Count occurrences of each line	sort access.log | uniq -c | sort -rn
wc	Word count: lines, words, characters	wc -l file.txt; wc -w essay.txt
cut	Extract specific columns from delimited files	cut -d':' -f1 /etc/passwd (usernames only)
tr	Translate or delete characters	echo 'hello' | tr 'a-z' 'A-Z' → HELLO

SECTION 4: Make Tool & Additional Shell Script Examples

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2079 Chaitra, 2075 Bhadra

Q8. What is the Make tool? How is it used in system administration and software development? [5]

Exam Ref: 2082 Kartik, 2080 Chaitra, 2079 Chaitra

Answer:

Make Tool — Introduction

Make is a build automation tool that automatically determines which parts of a large program need to be recompiled and executes the commands to rebuild them. It reads a special file called Makefile (or makefile).

Why Make is Needed:

- In large projects with hundreds of source files, recompiling everything on each change is slow
- Make tracks file dependencies and timestamps — only recompiles what changed
- Make ensures consistent build process across different environments

Makefile Structure

```
# Makefile basic structure
target: dependencies
    command_to_create_target       # MUST use TAB (not spaces)

# Variables
CC = gcc
CFLAGS = -Wall -g

# Build executable from object files
program: main.o utils.o
    $(CC) $(CFLAGS) -o program main.o utils.o

# Compile main.c to main.o
main.o: main.c
    $(CC) $(CFLAGS) -c main.c

# Compile utils.c to utils.o
utils.o: utils.c utils.h
    $(CC) $(CFLAGS) -c utils.c

# Clean up compiled files
clean:
    rm -f *.o program
```

Makefile Key Concepts

Concept	Explanation
Target	The file to be created (e.g., program, main.o)

Dependencies	Files that the target depends on; if any dependency is newer than target → rebuild
Recipe	Shell commands to build the target (indented with TAB)
Phony Targets	Targets that don't produce files (clean, install, all): .PHONY: clean install
Automatic Variables	$@ = target name, $< = first dependency, $^ = all dependencies
Pattern Rules	%.o: %.c allows generic rule for all .c files

Common Make Commands

```
make            # Build default target (first target in Makefile)
make clean      # Run the 'clean' target (remove compiled files)
make install    # Install built program (usually copies to /usr/local/bin)
make -n         # Dry run — shows commands without executing
make -j4        # Parallel build using 4 CPU cores
```

Q9. Write shell scripts to: (a) Check if a number is prime. (b) Print first N Fibonacci numbers. (c) Find largest of three numbers. [3+3+3]

Exam Ref: Various years — common shell scripting questions

Answer:

(a) Check if a Number is Prime

```
#!/bin/bash
echo -n 'Enter number: '
read n

if [ $n -le 1 ]; then
    echo '$n is NOT prime'
    exit
fi

is_prime=1
i=2
while [ $i -le $((n/2)) ]
do
    if [ $((n % i)) -eq 0 ]; then
        is_prime=0
        break
    fi
    i=$((i+1))
done

if [ $is_prime -eq 1 ]; then
    echo "$n is PRIME"
else
    echo "$n is NOT prime"
fi
```

(b) Print First N Fibonacci Numbers

```
#!/bin/bash
echo -n 'How many Fibonacci numbers? '
read n

a=0
b=1
echo -n 'Fibonacci: '
for ((i=0; i<n; i++))
do
    echo -n "$a "
    temp=$((a+b))
    a=$b
    b=$temp
done
echo
```

Output for N=8: Fibonacci: 0 1 1 2 3 5 8 13

(c) Find Largest of Three Numbers

```
#!/bin/bash
echo -n 'Enter three numbers: '
read a b c

if [ $a -ge $b ] && [ $a -ge $c ]; then
    echo "Largest = $a"
elif [ $b -ge $a ] && [ $b -ge $c ]; then
    echo "Largest = $b"
else
    echo "Largest = $c"
fi
```

QUICK REVISION: Chapter 9 — Key Points Summary

Topic	Key Points	Expected Marks
System Administrator	IT professional managing systems, security, users, backups, services, network	**2–3**
Roles & Responsibilities	10 responsibilities: User Mgmt, Security, Backup, Monitoring, Networking, Storage, Documentation	**3–6**
User Account Management	/etc/passwd, /etc/shadow, /etc/group; commands: adduser, userdel, usermod, passwd, chmod	**4–6**
Start/Shutdown	BIOS→GRUB→Kernel→init/systemd; Runlevels 0-6; systemd targets; shutdown commands	**3–5**
Shell Scripting	#!/bin/bash; variables ($var, $1, $?); if-elif-else; for/while loops; functions	**4–6**
Shell Script Examples	Factorial (while loop), Prime check, Fibonacci, Largest of 3 — must know pseudocode	**3–5**
AWK Tool	awk 'pattern {action}' file; Fields=$1,$2; NR, NF, FS; BEGIN/END blocks; great for CSV	**4–6**
grep, sed, AWK difference	grep=search; sed=replace; awk=field processing+computation	**2–3**
Search Tools	grep (pattern search), find (file search), locate (database search), which, whereis	**2–4**
Sort Tools	sort (-n numeric, -r reverse, -k column, -u unique), uniq (-c count), wc (-l lines), cut	**2–4**
Make Tool	Makefile: target:dependencies + recipe(TAB indent); CC,CFLAGS vars; make/make clean	**3–5**

Best of Luck! | TU IOE BCT | CT 656 OS | Chapter 9: System Administration